

AI ASSISTED CODING

2303A52346 BATCH 45

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

Scenario

You are developing a basic text-processing utility for a messaging application.

Task Description

Use GitHub Copilot to generate a Python program that:

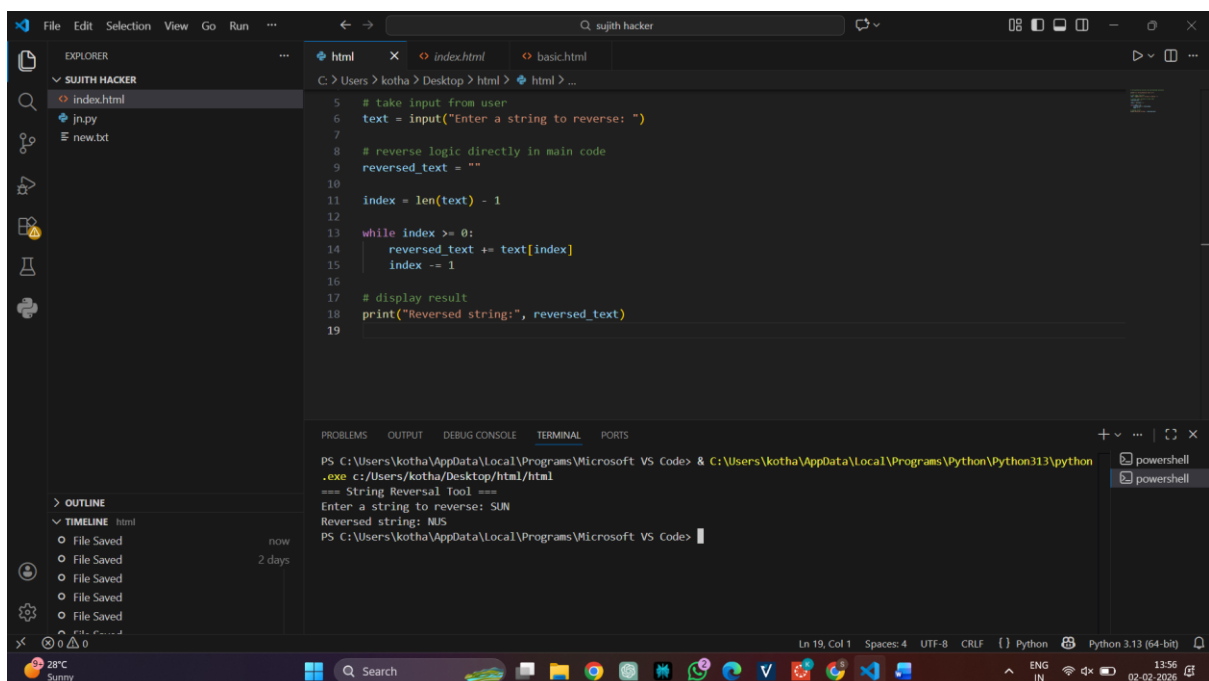
Reverses a given string Accepts user input

Implements the logic directly in the main code

Does not use any user-defined functions

❖ Expected Output

- Correct reversed string
- Screenshots showing Copilot-generated code suggestions
- Sample inputs and outputs



The screenshot displays the Visual Studio Code interface. The Explorer pane on the left shows a project named 'SUJITH HACKER' with files 'index.html', 'jupyter', and 'new.txt'. The main editor window shows a Python script in a file named 'html'. The script takes user input, reverses it using a while loop, and prints the result. The output pane at the bottom shows the command prompt execution: running the script with 'python .\exe c:/Users/kotha/Desktop/html/html', entering 'SUN' as input, and receiving 'NUS' as the reversed string output.

```
5 # take input from user
6 text = input("Enter a string to reverse: ")
7
8 # reverse logic directly in main code
9 reversed_text = ""
10
11 index = len(text) - 1
12
13 while index >= 0:
14     reversed_text += text[index]
15     index -= 1
16
17 # display result
18 print("Reversed string:", reversed_text)
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== String Reversal Tool ===
Enter a string to reverse: SUN
Reversed string: NUS
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
```

Task 2: Efficiency & Logic Optimization (Readability Improvement)

Scenario

The code will be reviewed by other developers.

Task Description

Examine the Copilot-generated code from Task 1 and improve it by:

Removing unnecessary variables

Simplifying loop or indexing logic

Improving readability

Use Copilot prompts like:

- “Simplify this string reversal code”
- “Improve readability and efficiency”

Hint:

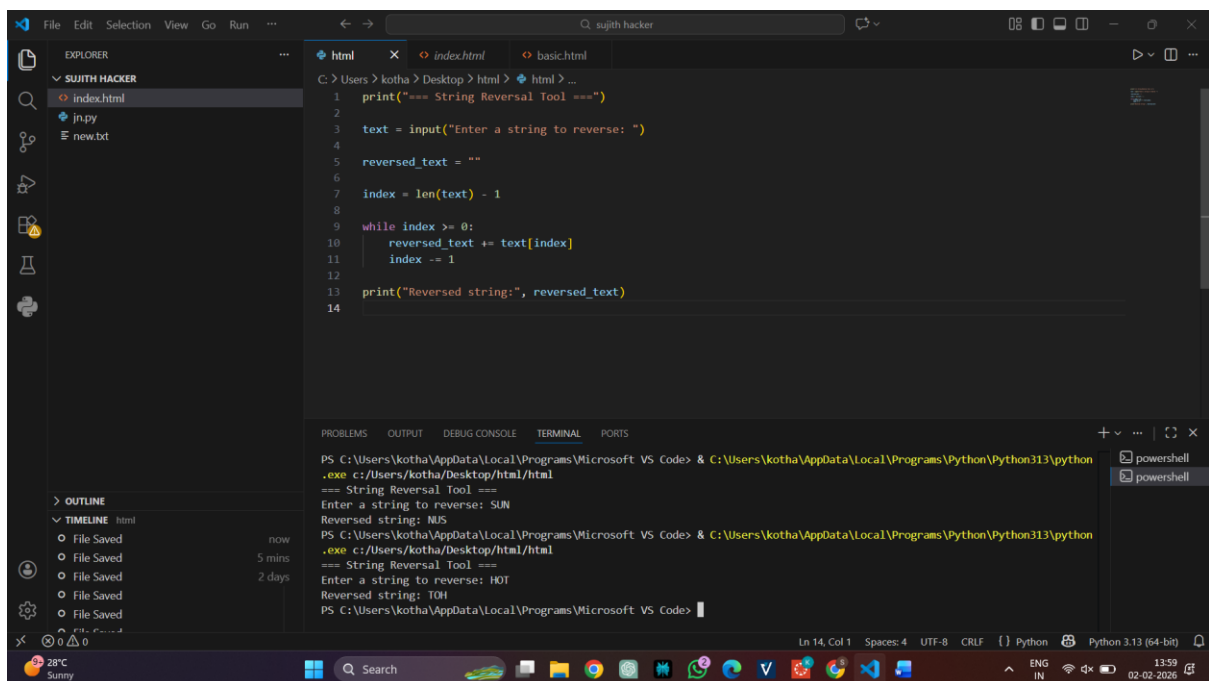
Prompt Copilot with phrases like

“optimize this code”, “simplify logic”, or “make it more readable”

❖ Expected Output

Original and optimized code versions

➤ Explanation of how the improvements reduce time complexity



The screenshot shows a Visual Studio Code editor with a Python file named `index.html` (though the extension is .html, the content is Python). The code implements a string reversal function using a while loop. The terminal output shows the script being executed twice: first with input "SUN" resulting in "NUS", and then with input "HOT" resulting in "TQH".

```
1 print("=== String Reversal Tool ===")
2
3 text = input("Enter a string to reverse: ")
4
5 reversed_text = ""
6
7 index = len(text) - 1
8
9 while index >= 0:
10     reversed_text += text[index]
11     index -= 1
12
13 print("Reversed string:", reversed_text)
14
```

Terminal Output:

```
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== String Reversal Tool ===
Enter a string to reverse: SUN
Reversed string: NUS
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== String Reversal Tool ===
Enter a string to reverse: HOT
Reversed string: TQH
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
```

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Scenario

The string reversal logic is needed in multiple parts of an application.

Task Description

Use GitHub Copilot to generate a function-based Python program that:

Uses a user-defined function to reverse a string

Returns the reversed string

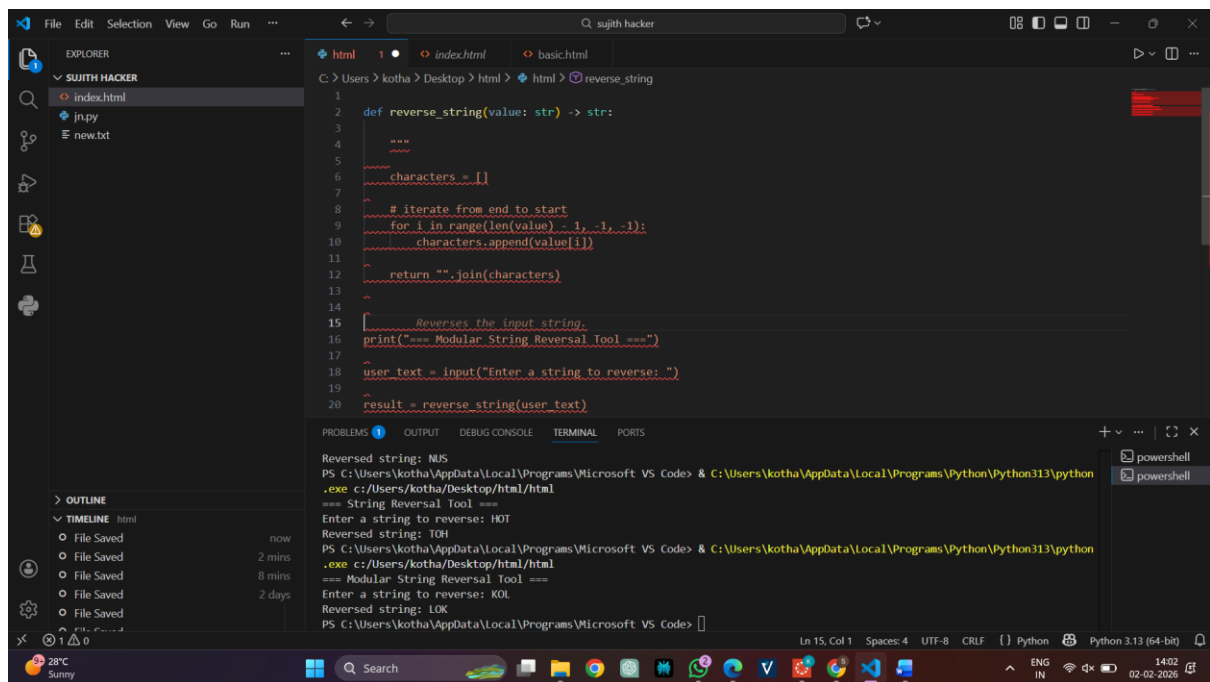
Includes meaningful comments (AI-assisted)

Expected Output

Correct function-based implementation

Screenshots documenting Copilot's function generation

Sample test cases and outputs



```
1
2 def reverse_string(value: str) -> str:
3
4     """
5     """
6     characters = []
7
8     # iterate from end to start
9     for i in range(len(value) - 1, -1, -1):
10         characters.append(value[i])
11
12     return "".join(characters)
13
14
15 [_____] Reverses the input string.
16 print("=== Modular String Reversal Tool ===")
17
18 user_text = input("Enter a string to reverse: ")
19
20 result = reverse_string(user_text)
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

Reversed string: NUS
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== String Reversal Tool ===
Enter a string to reverse: HDT
Reversed string: TOH
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== Modular String Reversal Tool ===
Enter a string to reverse: KOI
Reversed string: IOK
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

❖ Scenario

You are asked to justify design choices during a code review.

❖ Task Description

Compare the Copilot-generated programs:

➤ Without functions (Task 1)

➤ With functions (Task 3)

Analyze them based on:

➤ Code clarity

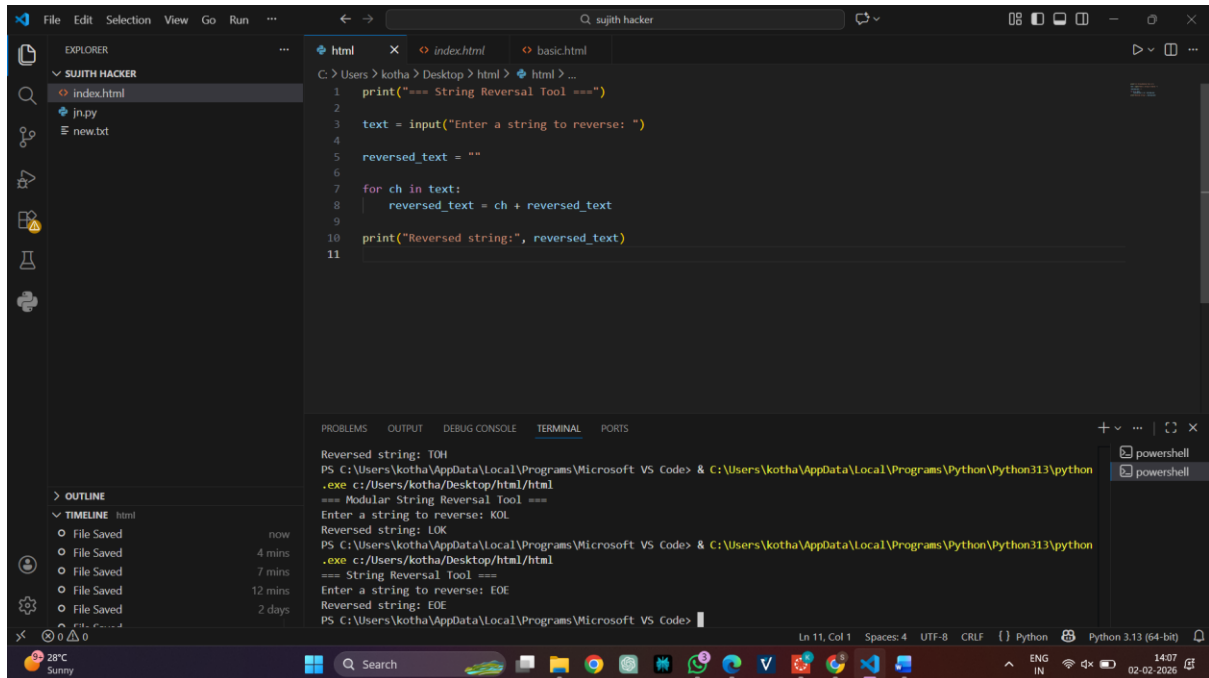
➤ Reusability

➤ Debugging ease

➤ Suitability for large-scale applications

❖ Expected Output

Comparison table or short analytical report



The screenshot shows a VS Code editor with a Python file named `index.html` (though the extension is .html, the content is Python). The code implements a string reversal tool using a loop. The terminal output shows the execution of the script, which prompts the user to enter a string to reverse. The output demonstrates the reversal of the strings "TOH", "LOK", and "EOE".

```
1 print("=== String Reversal Tool ===")
2
3 text = input("Enter a string to reverse: ")
4
5 reversed_text = ""
6
7 for ch in text:
8     reversed_text = ch + reversed_text
9
10 print("Reversed string:", reversed_text)
11
```

Terminal Output:

```
Reversed string: TOH
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== Modular String Reversal Tool ===
Enter a string to reverse: KOL
Reversed string: LOK
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code> & C:\Users\kotha\AppData\Local\Programs\Python\Python313\python
.exe c:/Users/kotha/Desktop/html/html
=== String Reversal Tool ===
Enter a string to reverse: EOE
Reversed string: EOE
PS C:\Users\kotha\AppData\Local\Programs\Microsoft VS Code>
```

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

❖ Scenario

Your mentor wants to evaluate how AI handles alternative logic paths.

❖ Task Description

Prompt GitHub Copilot to generate:

- A loop-based string reversal approach
- A built-in / slicing-based string reversal approach

❖ Expected Output

➤ Two correct implementations

➤ Comparison discussing:

- Execution flow
- Time complexity
- Performance for large inputs
- When each approach is appropriate

Note: Report should be submitted as a word document for all tasks in a

single document with prompts, comments & code explanation, and output

and if required, screenshots

